Structure of Programming Languages / Chapter 00

What Is a Programming Language?

Before building Vertex, decide what kind of thing a language is and what it takes to make one run.

Gregory S. DeLozier, PhD

01 / Shared rules

A token matters because people agree on what it means.

> A light, gesture, word, or keyword becomes part of a language when it stands for something beyond itself.

The raw token is not the interesting part. The shared rule that connects it to meaning is.

Traffic lights and referee signals are familiar languages: shared forms, shared meanings, real consequences.

Every language needs syntax and semantics.

Syntax

Which programs are **well-formed**?

```
x = 1 + 2;
print x;
```

Semantics

What do those programs **do**?

```
1 + 2  ->  3
x      ->  3
print x -> None
```

Side effect: `print x` writes `3`.

A parser can determine whether a program has legal form. It cannot, by itself, give that form behavior.

Text becomes structure before it becomes behavior.

Program
text
->
Tokens
->
Parse tree
or AST
->
Evaluation
or execution
->
Values,
output, effects

04 / Structure records precedence

Parsing makes the meaning in text explicit.

```
x = 1 + 2 * 3;
```

assign |- identifier: x `- + |- number: 1- * |- number: 2 `- number: 3
The abstract syntax tree stores the fact that multiplication happens inside addition.
Precedence is now data, not a guess from punctuation.

An evaluator runs the tree. A compiler translates it.

Evaluator / interpreter

Walk the AST and produce behavior now.

What happens when this node runs?

Compiler

Translate the AST into another form for later execution.

How can this meaning be expressed elsewhere?

JIT compilation and transpilation are variations on the same theme: preserve meaning while changing when or where a program runs.

06 / Programs remember

Variables require state, and state requires an environment.

```
x = 10;
y = x + 5;
```

environment |- x -> 10 `- y -> 15
Later, local scopes, function calls, and closures turn this simple lookup table into a chain of related environments.

A test suite is part of the language specification.

```
assert 2 + 3 * 4 == 14;
```

What it checks

The implementation produces the expected result.

What it defines

Multiplication binds more tightly than addition in Vertex.

08 / The course method

Vertex grows in snapshots, one pressure point at a time.

Arithmetic Expression parsing
Variables An environment
Conditionals Control-flow rules
Functions Call frames and returns
Closures Captured environments

Each new feature changes more than syntax. It creates new structural and semantic work in the implementation.

Chapter 00

Build the rules.
Then build the machine.

Next: give `2 + 3 * 4` the right structure and meaning.